

Backend ve Supabase Mulakat Soru-Cevap Notlari

Bu dokuman, HRCore projesinde Supabase backend tarafinda neler yapildigini isveren/mulakat formatinda anlatmak icin hazirlanmistir.

1. Bu projede neden ayri bir Node.js/Express backend yerine Supabase kullandin?

Cevap:

Bu projede MVP seviyesinde ama gercek backend mantigina sahip bir insan kaynaklari uygulamasini gelistirmek istedim. Supabase bana PostgreSQL database, Authentication, Storage ve Row Level Security ozelliklerini hazir verdigi icin ayri bir Express backend kurmadan backend ihtiyacini karsilayabildim.

Burada amac sadece hizli gelistirmek degildi. Supabase sayesinde iliskisel veritabani, rol bazli yetkilendirme, dosya saklama ve canli veri guvenligi gibi production'a yakin konulari da uygulamis oldum.

2. Supabase'i sadece database olarak mi kullandin, yoksa backend platformu olarak mi?

Cevap:

Supabase'i sadece database olarak kullanmadim. Bu projede Supabase backend platformu gibi calisti.

Kullandigim ana parcalar sunlardi:

- Supabase Auth ile kullanıcı girişi ve session yönetimi
- PostgreSQL tablolar ile uygulama verilerinin saklanması
- Supabase Storage ile çalışan dokümanları ve cihaz faturalarının saklanması
- Row Level Security ile rol bazlı veri erişimi
- Supabase JS Client ile React uygulamasının backend'e bağlanması

Yani uygulamanın backend tarafındaki temel katmanları Supabase üzerinden kuruldu.

3. `VITE_SUPABASE_URL` ve `VITE_SUPABASE_ANON_KEY` ne işe yarıyor?

Cevap:

`VITE_SUPABASE_URL`, Supabase projesinin API adresidir. React uygulaması hangi Supabase projesine bağlanacağını bu URL ile bilir.

`VITE_SUPABASE_ANON_KEY` ise frontend tarafından kullanılabilen public API key'dir. React uygulaması bu key ile Supabase'e istek atar.

Projede bu değerler `.env.local` dosyasından okunuyor ve `src/lib/supabaseClient.ts` içinde `createClient` ile Supabase client oluşturuluyor.

4. Anon key frontend'de gorunuyorsa bu guvenlik acigi degil mi?

Cevap:

Tek basina guvenlik acigi degil. Supabase'te anon key zaten frontend'de kullanilmak uzere tasarlanmistir. Buradaki asil guvenlik anon key'de degil, Row Level Security policy'lerindedir.

Eger RLS dogru yazildiyse, kullanıcı anon key ile Supabase'e istek atsa bile sadece yetkili oldugu satirlara erisebilir. Bu yuzden frontend'de service role key kullanmadim, sadece anon key kullandim.

5. Service role key nedir? Neden frontend'e koymamak gerekir?

Cevap:

Service role key, Supabase tarafinda RLS kurallarini bypass edebilen cok yetkili bir anahtardir. Admin seviyesinde islem yapabilir.

Bu yuzden kesinlikle frontend'e konmaması gerekir. Frontend kodu kullanıcının tarayicisinde calistigi icin bu key gorulebilir ve kotuye kullanılabilir. Service role key gerekiyorsa sadece server-side ortamda, ornegin Edge Function veya guvenli backend servisinde tutulmalidir.

6. Login akisi nasil calisiyor?

Cevap:

Kullanıcı email ve password girdiginde `supabase.auth.signInWithPassword()` fonksiyonu calisir. Supabase kullanıcıyı dogrular ve basariliysa session olusturur.

Sonra uygulama aktif session'i takip eder. `AuthProvider` icinde `getSession()` ile mevcut oturum okunur, `onAuthStateChange()` ile login/logout degisimleri dinlenir.

Login sonrasi `auth.users.id` alinir ve bu id ile `profiles` tablosundan kullanıcının uygulama rolu cekilir.

7. Supabase Auth ile `profiles` tablosu arasindaki iliski nedir?

Cevap:

Supabase Auth kullanıcının kimlik dogrulama tarafini yonetir. Ama uygulama icindeki rol bilgisi, tam ad ve aktiflik durumu `profiles` tablosunda tutulur.

Projede temel iliski su sekildedir:

```
auth.users.id = profiles.id
```

Yani Auth tarafindaki kullanıcı id'si ile `profiles` tablosundaki id aynidir. Bu sayede login olan kullanıcının profil ve rol bilgisi kolayca bulunur.

8. Neden kullanıcı rolunu direkt Supabase Auth icinde degil de `profiles` tablosunda tuttun?

Cevap:

Supabase Auth temel olarak authentication icindir. Uygulama seviyesindeki rol, aktiflik durumu ve profil bilgilerini ayrı bir tabloda tutmak daha esnek ve yonetilebilir bir yaklasimdir.

Bu sayede `admin_hr`, `manager`, `employee` gibi rolleri `profiles` tablosunda yonettim. Ileride rol degistirme, kullaniciyi pasife alma veya profil bilgisi guncelleme gibi islemler daha rahat yapilabilir.

9. `auth.users.id`, `profiles.id` ve `employees.profile_id` arasindaki iliski nedir?

Cevap:

Bu projede kullanıcı ile çalışan kaydı arasında zincirli bir ilişki kuruldu:

```
auth.users.id = profiles.id = employees.profile_id
```

`auth.users.id`, Supabase Auth kullanıcısıdır. `profiles.id`, bu kullanıcının uygulama profilidir. `employees.profile_id` ise bu profilin hangi çalışan kaydına bağlı olduğunu gösterir.

Bu ilişki sayesinde employee rolundeki kullanıcının kendi çalışan kaydını, izinlerini, cihazlarını ve dokümanlarını bulabiliyoruz.

10. Projede hangi roller var?

Cevap:

Projede üç temel rol var:

- `admin_hr`: Tüm çalışanları, izinleri, maaş kayıtlarını, cihazları ve dokümanları yönetebilir.
- `manager`: Kendi ekibindeki çalışanları ve ilgili iş süreçlerini görebilir.
- `employee`: Sadece kendi profilini, izinlerini, cihazlarını ve dokümanlarını görebilir.

Bu roller frontend'de ekran erişimi için, backend tarafında ise RLS policy'leri için kullanıldı.

11. `ProtectedRoute` ne işe yarıyor?

Cevap:

`ProtectedRoute`, React Router tarafında sayfa erişimini kontrol eden frontend katmanıdır. Kullanıcı login değilse login sayfasına yönlendirir. Login ise rolüne bakar ve ilgili sayfaya erişip erişemeyeceğini kontrol eder.

Orneğin sadece `admin_hr` rolünün girebileceği bir sayfaya employee rolundeki kullanıcı girmeye çalışırsa frontend bunu engeller.

12. `ProtectedRoute` gerçek güvenlik sağlar mı?

Cevap:

Hayır, tek başına gerçek güvenlik sayılmaz. `ProtectedRoute` kullanıcı deneyimini düzenler. Yani kullanıcının görmemesi gereken ekranları frontend'de gizler.

Gerçek veri güvenliği Supabase RLS policy'leri ile sağlanır. Çünkü frontend manipüle edilebilir ama Supabase tarafındaki RLS kuralları backend seviyesinde uygulanır.

13. Kullanıcı route guard'i bypass ederse veriye erişebilir mi?

Cevap:

RLS dogru yazildiyse erisemez. Kullanici tarayicida route'u degistirse veya manuel API istegi atsa bile Supabase her sorguda kullanicinin `auth.uid()` degerine gore izin kontrolu yapar.

Bu yuzden projede frontend role check ile yetinmedim, backend tarafinda RLS kullandim.

14. RLS nedir?

Cevap:

RLS, Row Level Security demektir. PostgreSQL'in satir bazli guvenlik sistemidir. Supabase bunu dogrudan kullanir.

RLS sayesinde her tablo satiri icin su sorular cevaplanir:

- Bu kullanıcı bu satiri gorebilir mi?
- Bu kullanıcı bu satiri ekleyebilir mi?
- Bu kullanıcı bu satiri guncelleyebilir mi?
- Bu kullanıcı bu satiri silebilir mi?

Bu projede employee'nin sadece kendi verisini, manager'in sadece kendi ekibini, admin/hr'nin ise tum verileri gormesi icin RLS mantigi kullanildi.

15. RLS ile frontend role check arasindaki fark nedir?

Cevap:

Frontend role check kullanıcı arayuzunu duzenler. Hangi menunun gorunecegi, hangi sayfaya girilecegi gibi konulari kontrol eder.

RLS ise backend seviyesinde veri erisimini kontrol eder. Kullanici frontend'i manipule etse bile RLS policy izin vermiyorsa veriyi okuyamaz.

Kisa ifade ile:

```
ProtectedRoute = UI/UX korumasi  
RLS = gercek veri guvenligi
```

16. Employee rolundeki kullanıcı baska calisanin verisini neden goremiyor?

Cevap:

Cunku RLS policy kullanicinin `auth.uid()` degerini calisan kaydindaki `profile_id` ile karsilastiriyor. Employee sadece kendi `profile_id` degeriyle eslesen satirlara erisebiliyor.

Mantik olarak:

```
employees.profile_id = auth.uid()
```

Bu eslesme yoksa Supabase ilgili satiri kullanıcıya dondurmuyor.

17. Manager sadece kendi ekibini nasıl görebiliyor?

Cevap:

`employees` tablosunda `manager_id` alanı var. Bir çalışanın hangi manager'a bağlı olduğu bu alanla tutuluyor.

Manager login olduğunda önce kendi employee kaydı bulunuyor. Sonra RLS policy, manager'ın sadece `manager_id` alanı kendi employee id'sine eşit olan çalışanlara erişmesine izin veriyor.

18. Admin/HR erişim mantığı nasıl?

Cevap:

Admin/HR rolü insan kaynakları yetkilisi gibi düşünüldü. Bu rol çalışan kayıtları, izin talepleri, maaş kayıtları, cihazlar ve dokümanlar üzerinde daha geniş yetkiye sahip.

RLS tarafında `profiles.role = 'admin_hr'` olan kullanıcılar için tüm satırlara erişim sağlanacak şekilde policy mantığı kuruldu.

19. `employees` tablosunda hangi bilgileri tuttu?

Cevap:

`employees` tablosunda çalışanın temel İK bilgileri tutuldu:

- Ad soyad
- Email
- Telefon
- Departman
- Pozisyon
- İşe başlama tarihi
- Çalışma durumu
- Temel maaş
- Profile bağlantısı
- Manager bağlantısı

Bu tablo uygulamanın merkezindeki ana iş verisini temsil ediyor.

20. `manager_id` alanı neden önemli?

Cevap:

`manager_id`, çalışanın hangi manager'a bağlı olduğunu göstermek için kullanıldı. Bu alan sayesinde ekip yapısı kuruluyor.

Manager rolündeki kullanıcı sadece kendi ekibindeki çalışanları görmeli. Bu sınırı kurmak için `manager_id` kritik bir alan.

21. Neden `devices` ve `device_assignments` diye iki ayrı tablo kullandın?

Cevap:

Çünkü cihaz bilgisi ile zimmet geçmişi farklı şeylerdir.

`devices` tablosu cihaz envanterini tutar. Örneğin laptop, telefon, marka, model, seri numarası ve durum bilgisi burada durur.

`device_assignments` ise cihaz kime, ne zaman atanmış, aktif mi, iade edilmiş mi gibi zimmet bilgilerini tutar.

Bu ayırım sayesinde bir cihazın geçmiş atamalarını kaybetmeden takip edebiliriz.

22. Bir cihaz çalışana atandığında backend tarafında ne oluyor?

Cevap:

Cihaz atama akisında iki işlem var:

1. `device_assignments` tablosuna yeni bir aktif atama kaydı eklenir.
2. `devices` tablosundaki cihaz durumu `assigned` olarak güncellenir.

Bu sayede hem cihaz kime atanmış bilgisi tutulur hem de cihaz envanterinde cihaz artık müsait görünmez.

23. Cihaz iade edildiğinde ne oluyor?

Cevap:

Cihaz iade edildiğinde aktif assignment kaydı `returned` durumuna çekilir ve `returned_at` tarihi yazılır. Ardından `devices` tablosundaki cihaz durumu tekrar `available` yapılır.

Bu şekilde cihaz yeniden atanabilir hale gelir ama eski zimmet kaydı da geçmiş olarak korunur.

24. İzin talebi oluşturma akisi nasıl?

Cevap:

Employee izin talebi oluşturduğunda `leave_requests` tablosuna yeni bir kayıt eklenir. Bu kayıta çalışan id'si, izin tipi, başlangıç tarihi, bitiş tarihi, toplam gün sayısı, açıklama ve durum bilgisi tutulur.

Yeni oluşturulan talepler varsayılan olarak `pending` durumunda kaydedilir.

25. İzin onaylanınca `reviewed_by` ve `reviewed_at` neden güncelleniyor?

Cevap:

Çünkü izin talebini kimin ve ne zaman değerlendirdiğini takip etmek gerekiyor. Bu hem iş süreci hem de audit/log mantığı açısından önemli.

Onay veya red sırasında aktif kullanıcı `supabase.auth.getUser()` ile alınır. Bu kullanıcının id'si `reviewed_by` alanına, işlem zamanı ise `reviewed_at` alanına yazılır.

26. Maaş kayıtlarını neden `salary_records` tablosunda tutun?

Cevap:

Maaş hesaplamaları dönemsel kayıtlardır. Her çalışan için ay ve yıl bazında maaş, ücretsiz izin günü, kesinti tutarı ve hesaplanan net tutar saklanmalıdır.

Bu yüzden `salary_records` tablosu kullanıldı. Böylece geçmiş maaş hesaplamaları korunur ve raporlama/CSV export gibi işlemler yapılabilir.

27. Maas hesaplama frontend'de mi backend'de mi yapiliyor? Production'da nasil gelistirirdin?

Cevap:

Bu MVP'de maas hesaplama frontend servis katmaninda yapiliyor ve sonuc Supabase'e kaydediliyor. Hesaplama basit oldugu icin bu MVP icin yeterliydi.

Ama production ortaminda maas gibi kritik bir hesaplamayi server-side yapmak daha dogru olurdu. Bunu Supabase Edge Function veya ayri bir backend endpoint'e tasirdim. Boylece kullanıcı frontend'i manipule etse bile nihai hesaplama guvenli ortamda yapilmis olurdu.

28. Dokuman upload akisini adim adim anlatir misin?

Cevap:

Dokuman upload akisi su sekilde calisiyor:

1. Kullanici dosya secer. 2. Dosya adi guvenli hale getirilir. 3. Storage path olusturulur. 4. Dosya Supabase Storage'a yuklenir. 5. Upload basarili olursa `documents` tablosuna metadata kaydi eklenir. 6. Liste yenilenir ve kullanıcı dokumani gorebilir.

Storage path formati:

```
employees/{employee_id}/{document_type}/{timestamp}_{safe_file_name}
```

29. Dosyayi Storage'a koyup neden ayrica `documents` tablosuna metadata yazdin?

Cevap:

Cunku Storage sadece gercek dosyayi tutar. Uygulama acisindan dosyanin hangi calisana ait oldugu, dokuman tipi, dosya adi, kim tarafından yuklendigi ve bucket/path bilgisi gibi metadata gerekir.

Bu yuzden:

```
Storage = gercek dosya  
documents tablosu = dosyanin uygulama kaydi
```

Bu ayrim filtreleme, listeleme, yetkilendirme ve signed URL uretme icin gerekli.

30. Storage bucket nedir? Bu projede hangi bucket'lari kullandin?

Cevap:

Storage bucket, dosyalarin saklandigi klasor benzeri alandir. Supabase Storage icinde farkli dosya gruplarini ayirmak icin bucket kullanilir.

Projede iki ana bucket kullandik:

- `employee-documents` : Calisan dokumanlari
- `device-invoices` : Cihaz faturalari

31. Storage path formatini neden bu sekilde sectin?

Cevap:

Path formatini calisan, dokuman tipi ve dosya adina gore organize ettim:

```
employees/{employee_id}/{document_type}/{timestamp}_{safe_file_name}
```

Bu yapi sayesinde dosyalar calisan bazli ayrilir. Dokuman tipi klasor gibi davranir. Timestamp ise ayni isimli dosyalarin cakismasini onler.

32. Signed URL nedir? Neden public URL kullanmadin?

Cevap:

Signed URL, private dosyaya gecici erisim saglayan imzali linktir. Projede dosya indirmek icin `createSignedUrl()` kullandim ve linki kisa sureli urettim.

Public URL kullansaydim dosyalar kalici olarak herkese acik hale gelebilirdi. IK dokumanlari hassas veri oldugu icin private storage ve signed URL daha guvenli bir yaklasimdir.

33. Private dosya erisimini nasil kontrol ettin?

Cevap:

Dosyalar private bucket mantiginda tutuldu. Kullanici dosya indirmek istediginde once `documents` tablosundaki metadata kaydina erisebiliyor mu diye RLS belirleyici olur.

Eger kullanıcı yetkiliyse uygulama ilgili bucket ve path için signed URL üretir. Yetkisiz kullanıcı metadata satirini goremedigi icin dosya path bilgisine ve signed URL akisina ulasamaz.

34. Dokuman silme akisi nasil calisiyor?

Cevap:

Projede once `documents` tablosundaki metadata kaydi siliniyor. Silme isleminde sonra gercekten satir silinmis mi diye `select('id')` ile kontrol ediliyor.

Eger metadata silindiyse sonra Supabase Storage tarafindaki dosya siliniyor. Storage tarafinda dosya zaten yoksa bu kritik hata olarak ele alinmiyor.

Bu yaklasimda kullanicinin silme yetkisi olup olmadigini RLS uzerinden once database tarafinda dogrulamis oluyoruz.

35. Upload sirasinda Storage basarili ama database insert basarisiz olursa ne yapiyorsun?

Cevap:

Bu durumda rollback mantigi uyguladim. Once dosya Storage'a yukleniyor. Sonra `documents` tablosuna metadata insert ediliyor.

Eger metadata insert basarisiz olursa, Storage'a yuklenmis dosya siliniyor. Boylece bucket icinde uygulamada karsiligi olmayan orphan dosya kalmasi engelleniyor.

36. RLS policy eksikligi yuzunden yasadigin bir problemi anlatir misin?

Cevap:

Dokuman silme ozelliginde employee kullanicisinin kendi dokumanini silmesi gerekiyordu. Ancak testte Storage dosyasi silinebilse bile `documents` tablosundaki metadata satiri silinmiyordu.

Script ciktisinda `cleanup_document=ERR no_rows_deleted` hatasini gorduk. Bu, sorgunun hata vermedigini ama RLS yuzunden hicbir satirin silinmedigini gosterdi.

Sorunun nedeni `public.documents` tablosunda employee icin DELETE policy eksikligiydi.

37. `cleanup_document=ERR no_rows_deleted` sana ne anlatti?

Cevap:

Bu sonuc bana DELETE sorgusunun calistigini ama RLS policy izin vermedigi icin herhangi bir satirin silinmedigini anlatti.

Yani problem frontend state problemi degildi. Backend tarafinda documents tablosu icin employee DELETE policy eksikti.

38. Bu problem nasil cozulmeli?

Cevap:

`documents` tablosuna employee'nin kendi dokumanini silebilmesini saglayan bir DELETE policy eklenmeli.

Ornek policy:

```
create policy "employees can delete own documents"
on public.documents
for delete
to authenticated
using (
  uploaded_by = auth.uid()
  or exists (
    select 1
    from public.employees e
    where e.id = documents.employee_id
    and e.profile_id = auth.uid()
  )
);
```

Bu policy ile kullanıcı kendi yukledigi veya kendi employee kaydına bagli dokumani silebilir.

39. Canli Supabase baglantisini nasil test ettin?

Cevap:

`scripts/` klasorunde canli kontrol scriptleri yazildi. Bu scriptler `.env.local` icindeki Supabase URL, anon key ve test kullanıcı bilgileriyle gercek Supabase projesine baglaniyor.

Ornek:

```
node scripts/supabase-live-check.mjs
node scripts/supabase-auth-check.mjs employee
```

Bu sekilde sadece uygulama ekranindan degil, dogrudan Supabase API uzerinden de baglanti ve yetki davranisi test edildi.

40. RLS sinirlarini nasil dogruladin?

Cevap:

`supabase-rls-boundary-check.mjs` scriptiyle admin, manager ve employee kullanicilariyla ayri ayri login olduk.

Sonra su senaryolari test ettik:

- Employee kendi kaydini okuyabiliyor mu?
- Employee baska calisani okuyamiyor mu?
- Manager kendi ekip uyesini okuyabiliyor mu?
- Manager takim disi calisani okuyamiyor mu?
- Admin tum calisan envanterini gorebiliyor mu?

Bu testler RLS sinirlarinin gercek ortamda calistigini gosterdi.

41. Supabase scriptlerini neden yazdin? Manuel test yeterli degil miydi?

Cevap:

Manuel test UI davranisini gormek icin faydali ama RLS gibi backend guvenlik konularinda yeterli degil. Cunku UI bazen veriyi gizleyebilir ama backend hala acik olabilir.

Scriptlerle dogrudan Supabase API uzerinden farkli rollerle sorgu attim. Bu bana RLS policy'lerinin gercekten dogru calisip calismadigini gosterdi.

42. Supabase schema degisikliklerini nereden yonettin?

Cevap:

Bu projede schema degisiklikleri Supabase Dashboard uzerinden yonetildi. Tablolar, policy'ler ve storage ayarlari dashboard/SQL Editor uzerinden kontrol edildi.

Repository icinde local migration sistemi yok. Bu MVP icin kabul edilebilir ama production ortaminda migration dosyalariyla ilerlemek daha dogru olur.

43. Bu projede migration sistemi var mi? Production icin nasil iyilestirirdin?

Cevap:

Bu projede local migration sistemi yok. Schema degisiklikleri Supabase Dashboard uzerinden yapildi.

Production icin Supabase CLI migration yapisi kurardim. Boylece tablo, policy ve index degisiklikleri kod gibi versiyonlanir, ekip icinde takip edilir ve CI/CD surecine dahil edilebilir.

44. Supabase kullanmanın avantajları neydi?

Cevap:

Supabase'ın en büyük avantajı MVP geliştirme hızını artırması ama aynı zamanda gerçek backend kavramlarını desteklemesiydi.

Avantajları:

- Hazır Auth sistemi
- PostgreSQL gibi güçlü ilişkisel database
- RLS ile satır bazlı güvenlik
- Storage ile dosya yönetimi
- React ile kolay entegrasyon
- Canlı API ve dashboard üzerinden hızlı kontrol

45. Supabase kullanmanın dikkat edilmesi gereken yanları neler?

Cevap:

Supabase kullanırken en dikkat edilmesi gereken konu RLS policy'leridir. Public schema API'ye açık olduğu için RLS kapalı veya yanlış yazılmışsa veri güvenliği riske girebilir.

Ayrıca service role key kesinlikle frontend'e konmamalıdır. Schema değişiklikleri de production'da dashboard üzerinden manuel değil, migration ile yönetilmelidir.

46. Bu backend yapısında en kritik güvenlik noktası nedir?

Cevap:

En kritik nokta RLS policy'lerinin doğru yazılmasıdır. Çünkü frontend tarafındaki buton, menu veya route kontrolleri asıl güvenlik değildir.

Kullanıcı hangi isteği atarsa atsin Supabase tarafında satır bazlı yetki kontrolü doğru çalışmalıdır. Bu yüzden RLS'i hem tasarladık hem de scriptlerle test ettik.

47. Production'a çıkacak olsan backend tarafında ilk neyi güçlendirirdin?

Cevap:

İlk olarak migration sistemini kurardım. Supabase CLI ile tablo, policy ve storage değişikliklerini versiyonlardım.

Sonra kritik iş mantıklarını Edge Function veya server-side backend'e taşıdım. Özellikle maaş hesaplama, toplu veri işlemleri, admin seviyesindeki operasyonlar ve audit log gibi konuları frontend'den uzaklaştırdım.

48. Edge Function kullanır mıydın? Hangi işlemleri taşırdın?

Cevap:

Evet, production ortamında kullandım.

Edge Function'a taşıyabileceğim işlemler:

- Maas hesaplama
- Kritik admin operasyonlari
- Toplu import/export islemleri
- Dosya isleme veya virus tarama gibi storage sonrasi islemler
- Audit log yazma
- Service role key gerektiren guvenli islemler

Boylece hassas is mantigi kullanicinin tarayicisinda degil, guvenli server-side ortamda calisir.

49. Bu projede backend tarafinda en cok ne ogrendin?

Cevap:

Bu projede backend'in sadece veri kaydetmek olmadigini daha iyi gordum. Auth, rol modeli, tablo iliskileri, RLS, storage metadata iliskisi ve canli test sureci birlikte dusunulmeli.

Ozellikle RLS policy eksikligini testle yakalamak benim icin onemliydi. Cunku frontend'de her sey dogru gorunse bile backend policy eksikse veri davranisi hatali olabiliyor.

50. Bu projeyi tek cumleyle backend acisindan nasil ozetlersin?

Cevap:

Bu projede Supabase'i backend platformu olarak kullanarak Auth, PostgreSQL, Storage, RLS ve rol bazli erisim katmanlarini kurdum; React tarafinda ise veri erisimini moduller bazinda service dosyalarina ayirarak daha okunabilir ve surdurulebilir bir yapi olusturdum.